

Guía Completa de Git y GitHub

Del primer *commit* a subir tu proyecto con `git push -u origin main`

Basada en el proceso real seguido en el proyecto **Pipelines**, incluyendo los errores que aparecieron en el camino (Git no reconocido, autenticación de GitHub, nombre de usuario incorrecto) y cómo se resolvieron.

1. ¿Qué es Git y por qué se necesita?

Git es un sistema de control de versiones: guarda el historial completo de cambios de tu código, para que puedas regresar a versiones anteriores, trabajar en equipo sin pisarte con otros, y respaldar tu proyecto.

GitHub es distinto de Git: es un servicio en la nube donde se alojan repositorios de Git, para poder compartirlos, respaldarlos fuera de tu PC, y colaborar. Git funciona sin GitHub (puedes usarlo 100% local); GitHub es el destino remoto opcional.

2. Instalación de Git en Windows

Opción A — con winget (recomendada)

```
winget install --id Git.Git -e --source winget
```

Opción B — descarga manual

Desde <https://git-scm.com/download/win>, ejecutar el instalador con las opciones por defecto.

Verificar la instalación

```
git --version
```

Nota: Si acabas de instalar Git y la terminal dice que no lo reconoce, **cierra la ventana de la terminal por completo y ábrela de nuevo** (o reinicia VS Code). Windows solo actualiza las rutas de programas (PATH) para ventanas nuevas, no para las que ya estaban abiertas. Esto fue exactamente lo que pasó al instalar Git en este proyecto.

3. Configuración inicial: tu identidad

Antes del primer commit, Git necesita saber quién eres (aparece como autor de cada commit). Se configura una sola vez por máquina, y aplica a todos los proyectos futuros:

```
git config --global user.name "tu-usuario"  
git config --global user.email "tu-correo@ejemplo.com"
```

Nota: El `user.name` es solo una etiqueta para tus commits — **no tiene que coincidir** con tu usuario real de GitHub. Son dos cosas distintas: una es cosmética (quién aparece como autor), la otra es tu cuenta real (la que determina la URL de tus repositorios).

4. Inicializar un repositorio local

Dentro de la carpeta de tu proyecto:

```
cd "C:\ruta\a\tu\proyecto"  
git init
```

Esto crea una carpeta oculta `.git` — ahí vive todo el historial. No se toca a mano.

5. El archivo `.gitignore`

Le dice a Git qué archivos **nunca** debe subir — no se excluyen solos, hay que listarlos explícitamente aquí. Es el lugar correcto para contraseñas, entornos virtuales y archivos generados.

```
.env/  
__pycache__/  
*.pyc  
  
# Secretos reales; solo .env.example se sube.  
.env  
  
# Archivos generados, no son código fuente.  
salidas/
```

***Nota:** El `.gitignore` en sí **sí se sube** a GitHub — su trabajo es excluir a los demás, pero él mismo es parte del código del proyecto.*

6. Primer commit

```
git add .  
git status # revisa qué se va a subir antes de confirmar  
git commit -m "Primer commit: descripción breve del proyecto"
```

- **git add .** — marca todos los archivos (menos los del `.gitignore`) para el próximo commit.
- **git status** — paso de seguridad: confirma que no se cuele ningún archivo con secretos.
- **git commit** — guarda esa versión en el historial local, con un mensaje descriptivo.

7. Crear el repositorio remoto en GitHub

- Entra a github.com → botón **New repository**.
- Ponle un nombre (ej. `pipelines`).
- Déjalo **vacío** — sin README, sin `.gitignore`, sin licencia — para evitar conflictos con tu primer push.
- Copia la URL HTTPS que te da (algo como `https://github.com/tu-usuario/tu-repo.git`).

Nota: Verifica el nombre de usuario exacto de tu cuenta de GitHub antes de usarlo en la URL (puede no coincidir con lo que tú "sientes" que es tu usuario, por ejemplo si tu cuenta terminó registrada como usuario-web en vez de usuario). Si te equivocas, no pasa nada grave — se corrige con el comando de la sección 8.

8. Conectar el repositorio local con GitHub

```
git remote add origin https://github.com/tu-usuario/tu-repo.git
git remote -v # confirma que quedó bien
git branch -M main # renombra la rama a "main"
```

Si ya agregaste el remoto pero con la URL equivocada, se corrige sin rehacer nada:

```
git remote set-url origin https://github.com/usuario-correcto/tu-repo.git
```

9. Autenticación: cómo suben tus credenciales

Git para Windows instala el **Git Credential Manager (GCM)**, que se encarga de guardar tu sesión de GitHub de forma seleccionable, sin que tengas que escribir usuario/contraseña cada vez.

Flujo normal (recomendado)

La primera vez que haces `git push`, se abre automáticamente una ventana del navegador pidiéndote iniciar sesión en GitHub (OAuth). Inicias sesión ahí normalmente (con tu 2FA si lo tienes activado), autorizas, y la terminal detecta que ya quedaste autenticado — continúa sola. Las siguientes veces, ya no te lo vuelve a pedir.

Alternativa: Personal Access Token (PAT)

Si el navegador no abre o prefieres no usar el flujo automático, puedes generar un token en GitHub → Settings → Developer settings → Personal access tokens, y usarlo como si fuera tu contraseña cuando la terminal lo pida.

***Nota:** Este flujo de autenticación (ventana de navegador) **necesita una terminal interactiva de verdad** — no funciona si el comando se ejecuta desde un proceso automatizado que no puede completar el login. Por eso los pasos de push siempre se deben correr directamente en tu propia terminal (la de VS Code o la de Windows), no delegados a un script sin supervisión.*

10. El primer push

```
git push -u origin main
```

git push — sube los commits locales al remoto.

- **origin** — el "apodo" del remoto que configuraste en el paso 8 (podrías tener varios, con distintos nombres).
- **main** — el nombre de la rama que subes.
- **-u (--set-upstream)** — le dice a Git "liga esta rama local con origin/main". Gracias a esto, ya no hace falta repetir "origin main" en los próximos push — basta con `git push` a secas.

11. Flujo de trabajo del día a día

Una vez configurado todo, el ciclo normal para subir cambios nuevos es:

```
git status # ¿qué cambió?  
git add . # marcar los cambios  
git commit -m "mensaje" # guardar la versión  
git push # subir a GitHub (ya sin -u ni la URL)
```

12. ¿Qué se repite si empiezo un proyecto nuevo?

Ya configurado (no se repite)	Se repite por cada proyecto nuevo
Tener Git instalado	git init
Identidad global (user.name / user.email)	Crear un repositorio nuevo y vacío en GitHub
Login de GitHub guardado en el Credential Manager	git remote add origin <url>
	.gitignore (adaptado al nuevo proyecto)
	Primer commit + git push -u origin main

13. Errores comunes (y cómo se resolvieron aquí)

"git : El término git no se reconoce..."

Causa: Git se instaló mientras la terminal ya estaba abierta.

Solución: cerrar y volver a abrir la terminal (o VS Code) para que cargue el PATH actualizado.

"fatal: could not read Username ... terminal prompts disabled"

Causa: se intentó hacer push/fetch desde un proceso no-interactivo, que no puede completar el login por navegador.

Solución: correr git push directamente en tu propia terminal interactiva.

Push a la URL equivocada / repo no encontrado

Causa: el nombre de usuario usado en la URL del remoto no coincidía con el usuario real de GitHub.

```
git remote set-url origin https://github.com/usuario-correcto/repo.git
```

"Se requiere autenticación" al usar contraseña normal

GitHub ya no acepta la contraseña de tu cuenta directamente para git — usa el login por navegador (GCM) o un Personal Access Token como se explicó en la sección 9.

14. Chuleta de comandos

Comando	Qué hace
git init	Crea un repositorio nuevo en la carpeta actual
git status	Muestra qué cambió desde el último commit
git add .	Marca todos los cambios para el próximo commit
git commit -m "..."	Guarda una versión en el historial local
git remote add origin <url>	Conecta el repo local con uno remoto
git remote -v	Muestra los remotos configurados
git remote set-url origin <url>	Corrige la URL de un remoto ya creado
git branch -M main	Renombra la rama actual a "main"
git push -u origin main	Primer push: sube y liga la rama local con la remota
git push	Sube cambios nuevos (una vez ligado con -u)
git pull	Trae cambios nuevos del remoto hacia tu copia local
git log --oneline	Historial resumido de commits